

AUTOKART

Simulation, Autonomy & Benchmark Development Report

31 July 2026 - 28 September 2026

Repository-grounded engineering status through commit 88e318a6

Purpose

Consolidated record of the FSDS simulator integration, ROS 2 sensor and autonomy software, experiment orchestration, benchmark UI, behavior-cloning work, mapped autonomy, observed failures, corrective actions, validation evidence, and planned implementation.

Simulation platform

FSDS v2.2 source integrated
ROS 2 Humble bridge
Standalone LinuxNoEditor
package

Sensors

LiDAR profiles and C++ processing
RGB/RGB-D depth profiles
IMU, TF and QoS adapters

Autonomy

Pure Pursuit and Stanley
Delaunay cone planning
SLAM teach/localize/repeat

Research tooling

Immutable Single Test manifests
Behavior-cloning pipeline
In-simulator benchmark menu

Scope: This report covers simulation and autonomy research software. Physical actuator firmware and electrical bench testing remain documented in the separate AutoKart RC development report used as this document's format reference.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | EXECUTIVE SUMMARY

Executive Summary

Engineering status as of 28 September 2026

The project has progressed from an FSDS checkout and manual ROS bridge bring-up into a substantial autonomy research platform. The current tree contains ten ROS 2 packages, 31 launch/operation scripts, 78 relevant source/configuration files, and approximately 9,093 lines of C++/Python source. It supports configurable sensor contracts, manual control, RGB-D visualization, VIO/VSLAM baselines, repeatable benchmark manifests, LiDAR cone planning, SLAM-based teach-and-repeat, and behavior-cloning data collection/inference infrastructure.

Confirmed progress

The canonical simulator and ROS settings are repository-local. LiDAR, RGB, metric depth, IMU and depth point-cloud topics have been observed. The standalone packaged simulator reaches TrainingMap, starts `AirSimGameMode`, and creates the native benchmark menu without Unreal Editor. Historical results include cone-free manual mapping laps and a cone-free SLAM Toolbox autonomous lap.

Primary unresolved issues

No current trained behavior-cloning model exists. The current AMZ teach-and-repeat artifact set is incomplete: the active sensor pose graph, cone map and Delaunay centerline are missing even though a closed route is present. Existing general-autonomy and local Delaunay trials did not complete a lap. Multi-vehicle racing and real-kart synchronization are designs, not validated functions.

Current evidence boundary

- A historical SLAM autonomous result completed one lap in 543.75 s with zero down/out cones; this proves termination, not competitive performance.
- Two manual mapping runs recorded laps near 456-460 s. The user-reported 44-50 s manual driving target is the performance reference, not the slow recorded runs.
- Behavior capture metadata reports 2,950 synchronized RGB/depth pairs with no write failures, but the CSV currently contains 2,900 samples; this mismatch must be resolved before training.
- The latest runtime smoke test verifies menu creation and absence of fatal/missing-package errors; it does not prove every button by automated click-through.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | PURPOSE, SCOPE & CLAIMS

Project Purpose & Research Scope

The intended product is an autonomy benchmark tool analogous to a graphics benchmark: choose a sensor and algorithm configuration, execute the same track protocol, and compare completion, lap time, cone contact, speed, stability, and compute behavior under a traceable manifest.

Single Test

Choice family	Implemented or planned options	Measured output
Sensors	A1M8, A3M1, FSDS 16-layer; HP60C, D415, D435, D455; combined LiDAR + RGB-D	Topic availability, rate, range contract, latency/freshness
Localization / SLAM	Sensor odometry, RGB-D odometry, RTAB-Map, SLAM Toolbox mapping/localization	Loop return, map availability, localization continuity
Planning	Reactive Delaunay, persistent mapped centerline, recorded route, behavior cloning, oracle debug baseline	Completion, path validity, interventions
Control	Pure Pursuit, Stanley; PID/state-estimator package work remains	Lap time, speed, cross-track behavior, cone contact

Multi Test

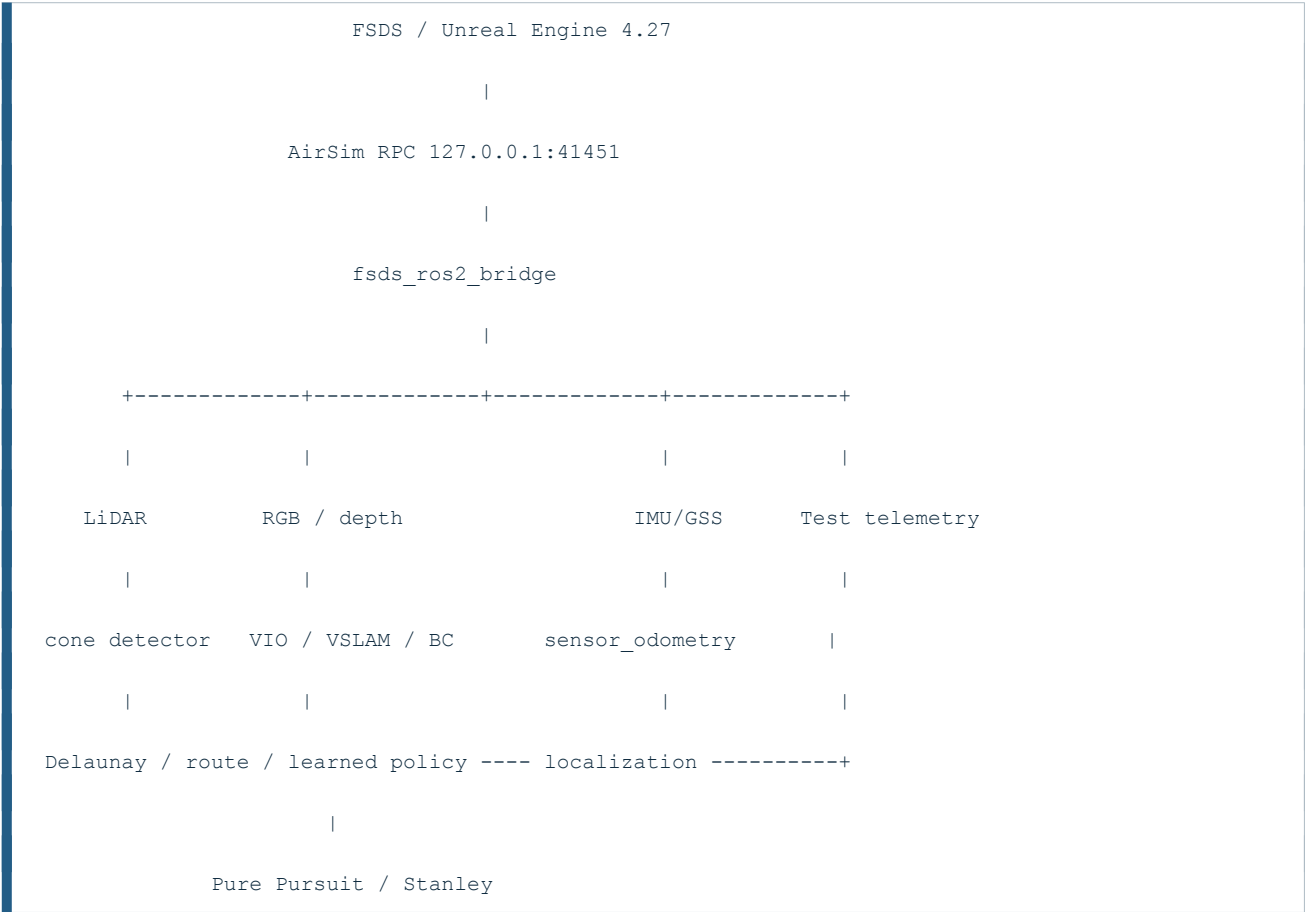
The planned mode runs three or four vehicles with different profiles, observes them from top/spectator views, and evaluates lap performance, overtaking and collision avoidance. This requires multi-vehicle namespacing, spawn control, per-vehicle bridges, command arbitration and race-state scoring. It is not yet implemented.

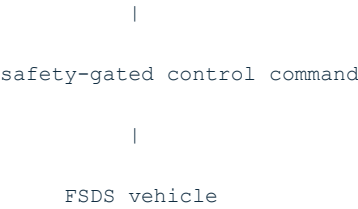
Research claim rule: `general_autonomy` consumes FSDS testing-only track and odometry topics. It is an oracle controller upper bound and debugging tool, not deployable autonomy. Sensor profiles reproduce externally visible data contracts, not full physical optics, sunlight response, projector behavior, USB timing or stereo physics.

Success definition

- 1. Reproducible manifest and artifact hashes.
- 2. One publisher owns vehicle control.
- 3. Safe disabled start and stale-input braking.
- 4. Repeated complete laps, not a single anecdotal success.
- 5. Zero cone contacts and explicit failure reporting.
- 6. Fair comparisons under equal speed and stopping rules.

Current Software Architecture





Benchmark control plane

```
In-simulator UI or single-test command

|

+-- prepare profile -> generated settings + immutable manifest

+-- restart FSDS      -> selected sensor contract becomes active

+-- bridge            -> ROS graph and TF

+-- launch disabled  -> readiness inspection / RViz

+-- arm               -> motion + timing

+-- stop              -> brake, cleanup, JSON result
```

Safety ownership: Controllers start disabled. Missing or stale path, speed, camera or telemetry inputs command braking. Launch assist is bounded and disarms after timeout. The scripts reject concurrent publishers on `/fdds/control_command`.

Canonical repository rule

The only active checkout is `/home/satvara/Github/Autonomous-kart-Sim`. FSDS source is under `FSDS/`, ROS 2 work under `FSDS/ros2/`, and the authoritative active settings file is `FSDS/settings.json`.

Development Timeline

Foundation through sensor and localization bring-up

Date	Change / test	Evidence and engineering result
31 Jul	Repository initialized.	5ceca7f7; starting point for the simulator benchmark project.
07 Aug	FSDS source, ROS 2 <code>fs_msgs</code> and bridge dependencies integrated.	db1df596, 0d4114b9, 134a7e5f.
08 Aug	AirSim math dependencies merged.	bb03f06b, d614d421.

09-11 Aug	Depth/RGB-D support, D455 profile, keyboard teleop, workspace relocation, VIO and VSLAM baselines.	3e18d82a through 8c5978b7. Camera, depth and odometry topics became operational.
12 Aug	Teleop shortcut, VIO/RGB-D merge and RTAB-Map correction.	22c26c60, 862940d4, db6c9ddf.
13 Aug	Initial simulation HUD test.	398c03aa, merged as 4b60f1e7.
14 Aug	Autonomy tests and LiDAR logic converted to C++.	0c918218, 9922c7c2, 8087e8c8.
17 Aug	Single Test manifest and session infrastructure.	5346af0d; reproducible settings/manifest/result contract introduced.

Date interpretation: Commit dates identify when software entered Git. Interactive simulator observations may have occurred before or after the commit on the same development day.

Development Timeline

Benchmarking, learned control, autonomy and packaging

Date	Change / test	Evidence and engineering result
19 Aug	Benchmark HUD iterations and UI patch.	b79e9c3b through a417b780.
20 Aug	Single Test autonomy passed software integration and sensor settings updated.	cc223e13, fa0298cb, 6385f889.
24 Aug	Custom map and Single Test update.	9cf30e2c; historical JSON run evidence preserved.
28 Aug	Behavior-cloning collection, training/export and C++ inference pipeline added.	92185f51, b3cbbad5.
29 Aug	Local planner and autonomy baseline extended.	461c6180, 2a2d32f8.
01-05 Sep	Repository-local settings, autonomy YAML, explicit arm/disarm and terminal launch consolidation.	157d97ed through a3e42483.
15 Sep	Autonomy benchmark UI, palette, general baseline, teach phase, persistent triangulation and working triangulation.	1263ea26 through 8813ff29.
17 Sep	RViz profile selection and normal standalone simulator package.	b3cfedcc, 88e318a6; packaged UI runtime smoke test passed.

Current Git state: Simulation-test is aligned with origin/Simulation-test and the working tree was clean during preparation of this report.

ROS 2 Package Inventory

Package	Current role	Status
fsds_ros2_bridge	AirSim RPC, sensors, vehicle commands, clock and TF.	Implemented
single_test	Profile validation, settings generation, manifests, lifecycle and result JSON.	Implemented
sensor_interface	Simulator/physical sensor adapters, currently IMU reframing.	Baseline
teleop	C++ keyboard control with arm/brake/quit controls.	Implemented
vio	RGB-D odometry and camera-frame IMU initialization contract.	Baseline
vslam	RTAB-Map RGB-D mapping and loop closure.	Baseline
slam	LaserScan conversion, sensor odometry, SLAM Toolbox and route recording.	Implemented baseline
autonomy	Cone detection, Delaunay planning, persistent map, route planning and controllers.	Implemented baseline
behavior_cloning	Demonstration capture, training/export, C++ inference and benchmark safety.	Pipeline implemented
fs_msgs	FSDS messages and services used by bridge and benchmark.	Implemented

Planned package separation

Package	Planned responsibility
interfaces	Common autonomy messages/services independent of FSDS.
state_estimator	Wheel odometry, chassis IMU and VIO fusion with covariance.
perception	Cones, obstacles and free-space estimation.
control	Controller plugins, command arbitration and real-vehicle interface.

FSDS Connection & Bridge

Validated connection path

```
FSDS standalone/editor -> AirSim RPC :41451 -> fsds_ros2_bridge

|

/flds/lidar/Lidar1, RGB, depth, IMU, GSS,

wheel states, signals, testing telemetry,

/flds/control_command and TF
```

The bridge was built for Ubuntu 22.04 / ROS 2 Humble in the kart-ros2-humble Distrobox. Connection was confirmed independently with nc -vz 127.0.0.1 41451 and through the ROS topic graph.

Repository-owned launch contract

```
cd /home/satvara/Github/Autonomous-kart-Sim

./FSDS.sh

# normal bridge / test entry points

FSDS/ros2/scripts/bridge-start

FSDS/ros2/scripts/single-test check
```

- Resolved configuration drift: Both simulator and bridge use the same repository-local FSDS/settings.json. Generated profiles may override it deliberately, but duplicate home-directory FSDS clones and symlinks are no longer part of the supported workflow.

Observed operational topics

/clock, /fsds/cam1/*, /fsds/cam2/*, /fsds/control_command, /fsds/gps, /fsds/gss, /fsds/imu, /fsds/lidar/Lidar1, /fsds/lidar/Lidar2, /fsds/signal/go, /fsds/testing_only/odom, /fsds/testing_only/track, /fsds/wheel_states, /tf_static

- Environment boundary: Host and Distrobox shells must not source each other's generated setup paths. ROS commands run inside the Humble container; FSDS runs as the host graphical application and communicates over loopback.

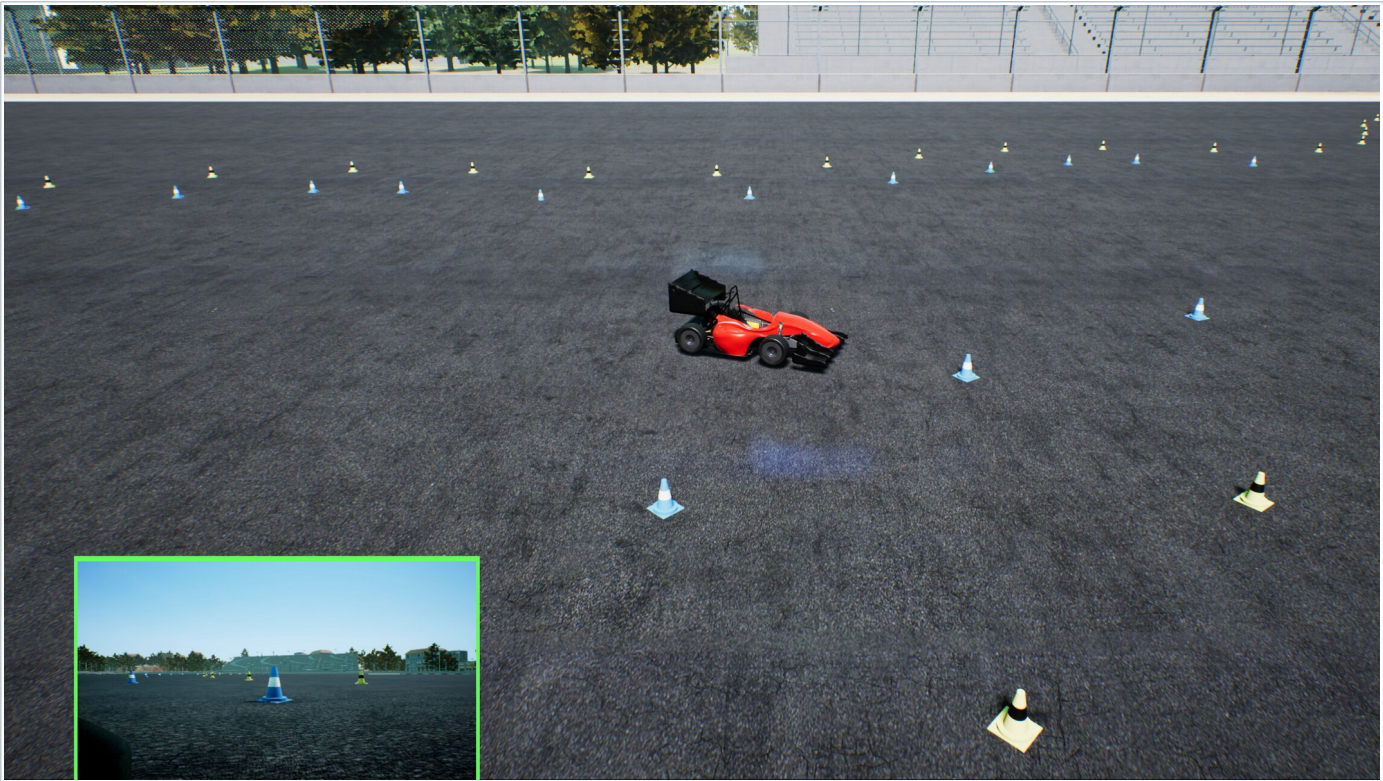
Configurable Sensor Contracts

LiDAR profiles

Profile	Range	Scan contract	Use
A1M8	12 m	1 layer, 1,600 points, 5 Hz, 360 degrees	Low-cost 2D SLAM comparison
A3M1	25 m	1 layer, 1,600 points, 10 Hz, 360 degrees	Mapped SLAM benchmark
FSDS 16	100 m	16 layers, 4,096 firings, 10 Hz, forward 180 degrees	Dense development/autonomy profile

RGB-D profiles

Profile	Resolution / rate	FOV / range	Limit
YDLIDAR HP60C	640x480 at 20 Hz	RGB 80.9°, depth 73.8°, 0.2-4.0 m	No physical structured-light noise/projector model
RealSense D415	1280x720 at 30 Hz	RGB 69.4°, depth 65°, 0.16-10 m	No stereo/shutter artifact model
RealSense D435	848x480 at 30 Hz	RGB 69°, depth 87°, 0.105-10 m	No physical baseline/projector behavior
RealSense D455	848x480 at 30 Hz	RGB/depth 86°, 0.4-6.0 m, nominal 95 mm baseline	FSDS renders depth; baseline does not create disparity



Depth Camera Integration

Implemented processing

- Separate front RGB and metric depth captures with camera calibration topics.
- Depth point-cloud conversion through `depth_image_proc`.
- RGB/depth frame alignment and optical-frame TF correction.
- Best Effort visualization QoS for simulator sensor publishers.
- RViz layouts for generic sensors, behavior cloning, general autonomy and AMZ planning.

Key transform

```
fsds/FSCar -> fsds/front_rgb

translation: [0.800, 0.000, 0.800]

RPY: [-90 degrees, 0 degrees, -90 degrees]

camera optical convention: +Z forward, +X right, +Y down

vehicle convention:      +X forward, +Y left, +Z up
```

Errors resolved

Symptom

Depth image black/blank

RGB and depth frame IDs differed

RViz received no point cloud

Cloud appeared vertical/perpendicular

D455 limitation: The FSDS chassis IMU can be republished in the camera frame for timing and gravity initialization, but it does not reproduce camera lever-arm acceleration or the D455's physical camera-mounted IMU.

Visual Odometry & Mapping

VIO baseline

The `vio` package launches synchronized RGB-D odometry and an IMU adapter, publishing `/localization/vio/odometry`. Recorded console evidence showed repeated RTAB-Map odometry updates with roughly 700-800 visual correspondences and approximately 0.10-0.13 s delay.

What this proves: RGB-D feature tracking, synchronized depth and odometry publication operated in the simulator. It is a visual-inertial initialization baseline, not a final tightly coupled estimator.

VSLAM baseline

RTAB-Map receives RGB, depth, calibration, VIO odometry and IMU initialization. Loop-closure views and a 3D cloud were observed. Early 3D-map displays looked like dense planar cells because the depth cloud was being visualized without a useful persistent trajectory/map interpretation.

```
RGB + registered metric depth + camera_info
    |
    | rgbd_odometry + IMU init
    |
    | RTAB-Map graph / loop closure
    |
    | map -> vio/odom -> fsds/FSCar
```

Remaining work

- Quantify drift against evaluation-only ground truth.
- Run repeated loop closures and record graph corrections.
- Separate mapping and localization trials.
- Add a proper fused state estimator and calibrated covariance.
- Validate the same frame/timestamp contract on the physical RGB-D camera.

Interpretation rule: A visually dense RTAB cloud is not by itself evidence of accurate localization. Map consistency, loop-closure residuals, trajectory error and repeatability must be measured.

LiDAR SLAM & Saved-Route Autonomy

```
/fsds/lidar/Lidar1 -> LaserScan -> SLAM Toolbox -> slam/map

IMU + GSS -> sensor_odometry -> sensor/odom -> fsds/FSCar

saved route + localized pose -> route_planner -> Pure Pursuit / Stanley
```

Implemented features

- Sensor-derived odometry avoids testing-only odometry in deployable pipelines.
- Mapping and saved-graph localization launch modes.
- C++ route recorder sampling at 0.40 m.
- Geometric loop-return gate with minimum travel and heading agreement.
- Pure Pursuit and Stanley profiles using the same map and route.
- Pose-graph, occupancy map and route artifact checks before motion.

Historical run evidence

Run	Outcome	Lap	Distance	Cones
Manual mapping	Manual stop	456.16 s	504.11 m	0
Manual mapping	Lap target	459.61 s	461.77 m	0
SLAM Toolbox track drive	Lap target	543.75 s	543.63 m	0

Performance warning: The 543.75 s autonomous lap and 456-460 s mapping laps are not acceptable speed references when the manual track can be driven in roughly 44-50 s. They only show that the pipeline reached a lap event without cone contact.

Current artifact gate: route_sensor_slam.json exists and is closed with 808 points, but the active track_sensor.posegraph/.data/.pgm/.yaml files are absent; only backups remain. Loaded-map autonomy must not start until the complete active artifact set is restored.

Reactive Delaunay Autonomy

Implemented algorithm

1. Filter and cluster LiDAR returns into cone candidates.
2. Separate left/right boundaries.
3. Build Bowyer-Watson Delaunay triangulation.
4. Retain opposite-boundary edges inside valid track-width limits.
5. Use edge midpoints as centerline gates.
6. Connect gates sharing triangles and grow alternatives with beam search.
7. Score heading smoothness, width consistency, spacing, horizon and color ordering.
8. Publish selected path to the safety-gated controller.

```
/fsds/lidar/Lidar1
-> cone_detector
-> /autonomy/cones/left + /autonomy/cones/right
-> local_planner
-> /autonomy/delaunay_midpoints + /autonomy/local_path
-> pure_pursuit
-> /fsds/control_command
```

Visualization contract

Visualization	Meaning
n	
Dark lines	Raw Delaunay triangulation.
Pale blue	Valid cross-track gates.
Green	Candidate centerlines explored by beam search.
Thick red	Lowest-cost candidate.
Magenta path	Smoothed path consumed by controller.

Historical trial outcome

Three `delaunay_phase1` results ended by manual stop or 180 s time limit. Distances were 0-6.36 m; none completed a lap. These files are useful failure evidence but not proof of autonomous operation.

Fallback behavior: If safe Delaunay gates are unavailable, the planner tries paired cones and then conservative single-boundary offset. If no method is safe, it publishes an empty path and the controller brakes.

Persistent Cone Map & Centreline

Intended integrated sequence

- 1. Prepare `amz_style` before FSDS starts.
- 2. Drive one or two manual laps with C++ keyboard control.
- 3. Transform cone observations and Delaunay midpoints into `slam/map`.
- 4. Associate persistent left/right landmarks while SLAM corrects pose.
- 5. Accept return only after travel distance, radius and heading gates pass.
- 6. Save pose graph, cone map, teaching route and ordered Delaunay centerline.
- 7. Stop teleop publication, launch localization, wait for readiness and arm replay.

```
FSDS/ros2/scripts/single-test prepare amz_style

# restart/start FSDS

FSDS/ros2/scripts/single-test teach-run --laps 1

# required replay artifacts

track_sensor.posegraph + track_sensor.data

track_sensor.pgm          + track_sensor.yaml

cone_map.json             + centerline_delaunay.json

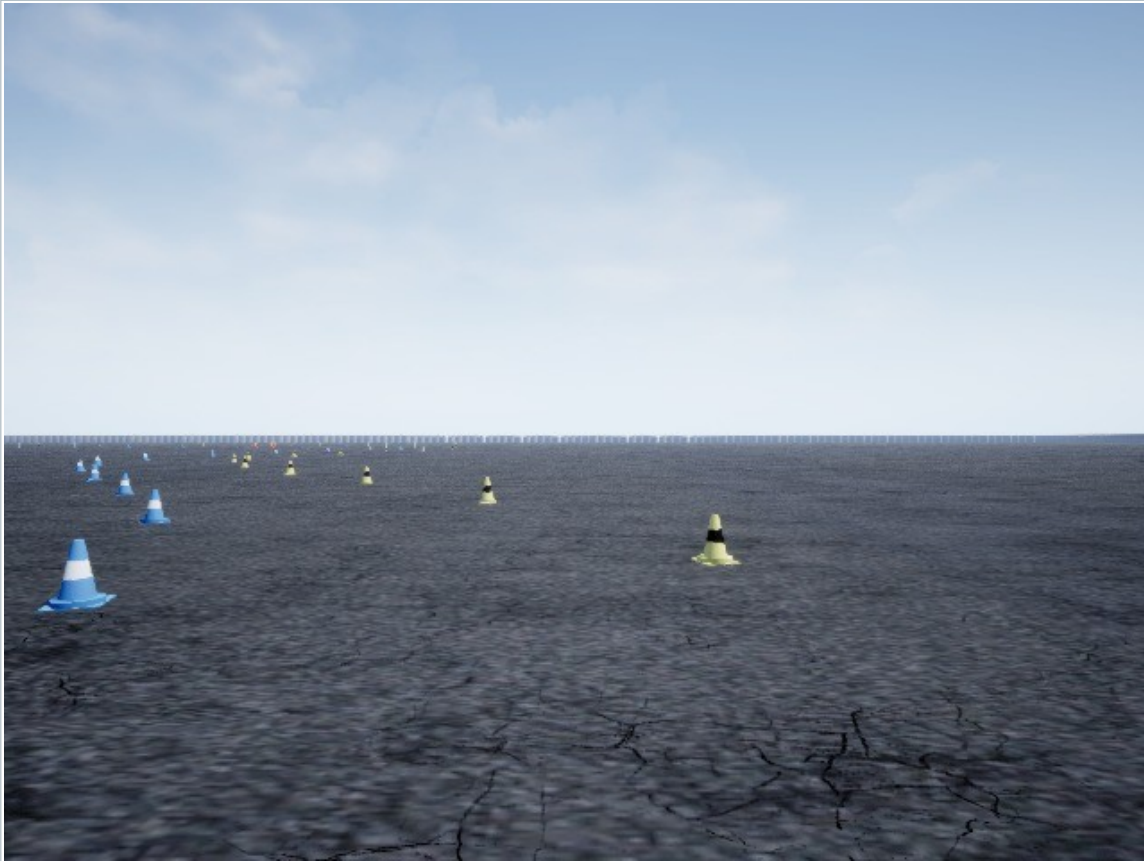
route_sensor_slam.json
```

Current repository evidence

Artifact	Current state
Closed sensor-SLAM teaching route	Present; 808 points, geometric loop closure.
Active sensor pose graph/map	Missing; timestamped backups exist.
Persistent cone map	Missing.
Delaunay centerline	Missing.
Autonomous replay result	Not demonstrated.

Current conclusion: The teach-and-repeat code path is implemented, but the repository is not presently ready to launch loaded AMZ autonomy. A fresh, clean manual teaching lap must generate and validate all required artifacts first.

RGB / RGB-D Imitation Learning



Example frame from the current HP6oC-style demonstration capture.

Implemented pipeline

Stage	Implementation
Collect	C++ synchronized RGB, metric depth, speed and expert command recorder.
Validate	Timestamp, range, resolution, encoding and file-existence checks.
Train	PyTorch RGB and RGB-D models; split by complete run, not adjacent frames.
Export	ONNX plus immutable preprocessing/model metadata.
Shadow	Publish prediction separately while expert retains control.
Drive	C++ OpenCV-DNN inference with stale-input brake, limits, launch assist and disabled start.
Benchmark	60 s limit, lap/cone stop and JSON result.

Current dataset state

- One run directory exists.
- 2,950 RGB files and 2,950 depth files exist; metadata reports 2,950 samples and zero write failures.
- The CSV has 2,900 data rows, creating a 50-sample accounting mismatch.
- No trained `model.onnx` or `model.json` exists.

Training gate: One run is insufficient for a defensible train/validation split. Resolve the count mismatch, collect at least 12 clean independent laps plus recovery demonstrations, reserve complete held-out runs, and validate in shadow mode before closed-loop control.

Reproducible Experiment Framework

Executable profiles

Profile	Pipeline
AMZ style	LiDAR cones -> Delaunay -> Pure Pursuit
Behavior cloning	RGB-D + speed -> learned C++ policy
SLAM	SLAM Toolbox graph + recorded route -> Pure Pursuit
SLAM + Stanley	Same graph/route -> Stanley
General autonomy	Testing-only track/odom -> Pure Pursuit

Lifecycle contract

```
single-test prepare PROFILE

-> validate YAML + sensor catalog

-> generate settings.json + immutable manifest

-> apply canonical FSDS/settings.json


single-test launch PROFILE

-> bridge if required

-> stack starts DISABLED

-> readiness gates

-> operator confirmation
```

```
-> arm + time + record  
  
-> stop/brake + result JSON
```

Recorded metrics

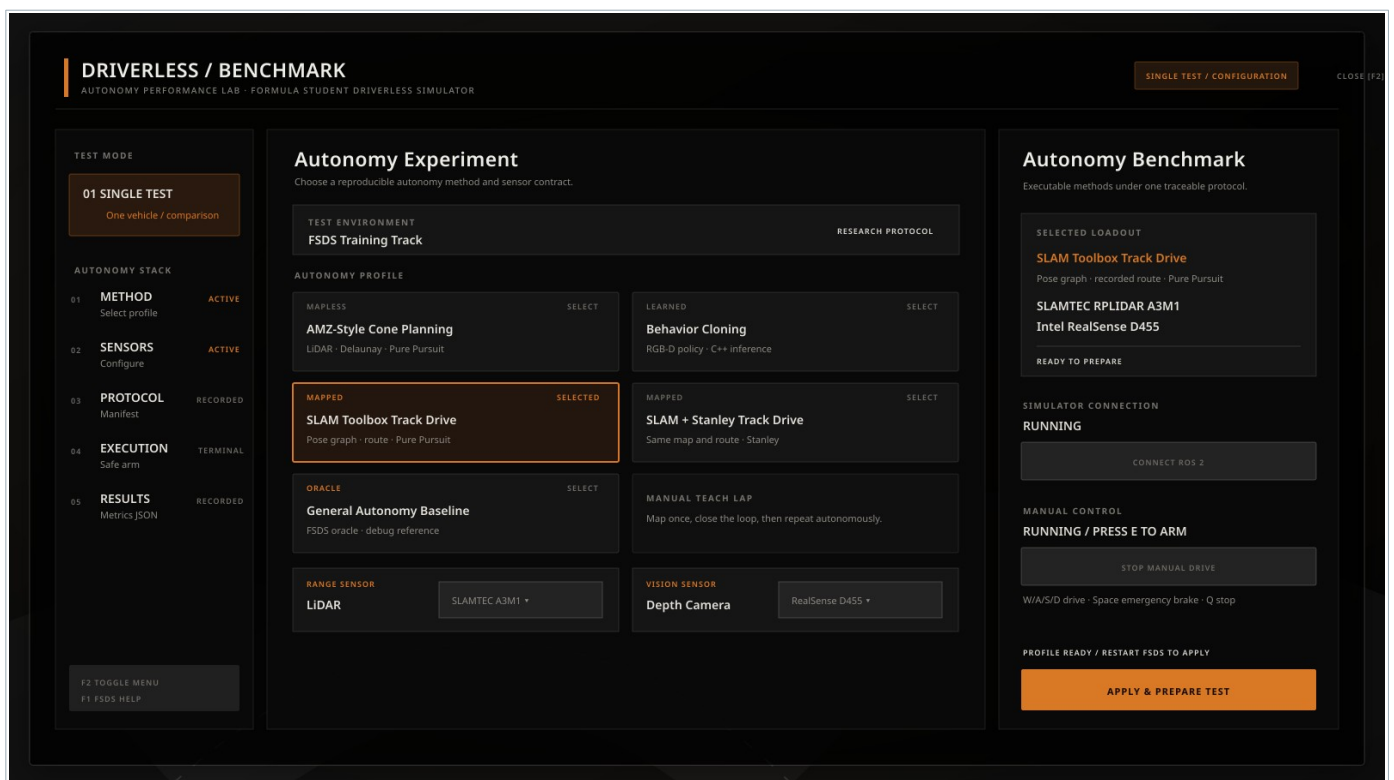
Duration, integrated distance, average and maximum speed, FSDS lap times, cone contacts, completion reason, exact sensor/algorithm selection, map/model paths and research metadata.

Research strength: Every run can carry its complete immutable manifest. This prevents a lap result from becoming detached from the sensor profile, controller, map/model and privileged-data status that produced it.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | IN-SIMULATOR UI

Benchmark Menu / HUD



Current black/grey/white/orange benchmark menu preview.

Implemented controls

- Autonomy profile cards for AMZ, behavior cloning, SLAM + Pure Pursuit, SLAM + Stanley and general baseline.
- LiDAR and depth-camera selectors.
- ROS bridge start/status.
- Manual keyboard-drive start/stop.
- Apply/prepare profile through repository scripts.

- Profile-specific RViz launcher.
- F2 menu toggle and runtime status text.

Runtime evidence: The packaged TrainingMap log reached `AirSimGameMode` and printed `Benchmark menu: added native UMG host to viewport with no fatal or missing runtime package error.`

Validation limit: Environment paths and all four referenced executable scripts were verified, but automated click-through of every button was not performed. Full acceptance still requires pressing each control in the packaged build and verifying its process/result state.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | STANDALONE SIMULATOR

Normal Launch Without Unreal Editor

Implemented package

```
cd /home/satvara/Github/Autonomous-kart-Sim

./FSDS.sh

# rebuild after Unreal/C++ HUD changes

FSDS/scripts/fsds-package --development
```

The package is a LinuxNoEditor build, approximately 2.5 GB, with a repository-root launcher. It opens the normal FSDS menu; selecting TrainingMap loads the simulator and benchmark HUD. No FSONline Editor or Unreal Editor is needed during routine operation.

Packaging fixes

Failure

Non-editor C++ build failed in Chaos.

HUD header failed outside unity/editor build.

TrainingMap package crashed loading car.

Weather classes missing at runtime.

Final runtime evidence

- Package build completed successfully.
- Pak inspection found TrainingMap, MainMenuLevel, TechnionCarPawn and wheel materials.
- Runtime reached TrainingMap and created the benchmark menu.
- No fatal exception, SIGABRT, missing asset class or missing package remained.

Nonfatal inherited warnings: Main-menu focus, start-line zero scale, camera root component, Pawn EnableInput and initial weather actor-array warnings remain. They did not stop the validated runtime, but should be cleaned before a polished release.

Teleoperation, Controllers & Motion Safety

Manual control

Key	Function
E	Arm/disarm manual control.
W / S	Throttle / reverse or deceleration behavior.
A / D	Steer left / right.
Space	Emergency brake.
Q	Stop teleop process.

Controller safety behavior

- Disabled by default and armed through service/script confirmation.
- Fresh path and GSS speed required.
- FSDS GO signal required where configured.
- Stale/missing input produces full brake.
- First new cone contact can terminate benchmark.
- Full-lap event stops and writes result.
- Concurrent command publishers are rejected.

Static-friction launch assist

```
launch throttle:      0.55
pulse duration:       0.10 s
release speed:        0.10 m/s
failure timeout:      0.75 s

if speed reached -> normal tracking
```

```
if timeout reached -> full brake + disarm; no automatic retry
```

This bounded kick addresses observed FSDS static resistance without globally changing Unreal friction. It must be tuned from command/GSS recordings.

Safety rule: Never run keyboard teleop and an autonomous controller simultaneously. Exactly one publisher must own `/fsds/control_command`.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | RECORDED TEST RESULTS

Result Evidence in Repository

Family

Manual mapping

SLAM route drive

General oracle

AMZ local Delaunay

Behavior cloning

Performance reference

The user reports a manual TrainingMap lap around 44 seconds. A realistic first autonomy gate is a complete cone-free lap under 60 seconds. Existing 456-544 s artifacts must not be used as the optimization reference.

Result-quality requirements

- At least ten attempts per final profile.
- Median and dispersion, not only fastest lap.
- Completion ratio and every failure reason.
- Zero cone contact / off-track acceptance gate.
- Fixed track, seed, speed limit and reset pose.
- Map/model hash in every result.

Data hygiene: Several August result files predate the current richer schema and have no explicit state field. Preserve them as historical evidence, but generate fresh results under the current manifest contract for comparisons.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | CURRENT ARTIFACT INVENTORY

Maps, Routes, Datasets & Builds

Artifact
Generic SLAM pose graph
Sensor-SLAM route
Sensor-SLAM active graph
Invalid duplicate-TF route
AMZ cone map / centerline
BC dataset run-001
BC models
Standalone simulator
Single Test manifests
Storage note: Generated datasets, maps, build/install/log directories, results and packaged binaries are intentionally not source-controlled. Reports must therefore record hashes and archive locations if results need long-term reproducibility.

Immediate artifact recovery

1. Archive current generated/maps/TrainingMap. 2. Run a fresh one-lap AMZ teach session. 3. Require all seven replay files to be non-empty. 4. Inspect persistent cones/centerline in RViz. 5. Only then launch autonomous replay.

Bring-up Errors

Date
07 Aug
07 Aug
07-11 Aug
09 Aug
09-11 Aug

11 Aug

11 Aug

12 Aug

Autonomy, UI & Packaging Errors

Date	Error / symptom
Aug-Sep	Vehicle did not move at low throttle.
15 Sep	Launch rejected existing control publisher.
15 Sep	Local cones/path did not leave a persistent track.
15 Sep	Route contained duplicate-TF corruption.
Aug-Sep	UI/editor repeatedly crashed or project path not found.
17 Sep	Packaged game compile failed.
17 Sep	Package crashed on TrainingMap.
17 Sep	Weather actor/menu packages missing.
Current	BC metadata and CSV counts differ by 50.
Diagnostic principle: Build success is not runtime success. The package only became valid after pak inspection and a real TrainingMap smoke test exercised dynamic asset loading and HUD creation.	

Current Validation Matrix

Subsystem
FSDS RPC / ROS bridge
LiDAR / RGB-D / IMU
Depth point cloud / TF
VIO / RTAB VSLAM
Single Test

General autonomy

Reactive Delaunay

AMZ teach/repeat

SLAM route autonomy

Behavior cloning

Benchmark UI

Standalone build

Historical software test note: A prior project validation run recorded 588 passing tests. This report does not represent that as a fresh 28 September ROS/container rerun; current live acceptance still requires the functioning Humble environment.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | COMPLETION ESTIMATE

Implementation Readiness Estimate

Engineering estimate, not formal test coverage

Area

Simulator/bridge foundation

Sensor configuration/visualization

Single Test orchestration

Benchmark UI

SLAM mapped autonomy

AMZ teach/repeat

Behavior cloning

General deployable autonomy

Multi-vehicle racing

Simulation-to-real synchronization

Overall interpretation: The platform/infrastructure is substantially ahead of autonomy performance. The next milestone should be one reproducible full-lap autonomy profile, not additional breadth.

Recommended focus

Freeze sensor/profile selection, regenerate one high-quality map, validate AMZ or SLAM replay at conservative speed, and only then optimize toward the sub-60-second target.

Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | IMPLEMENTATION ROADMAP

What Will Be Implemented

Phase

1. Artifact recovery
2. Conservative replay
3. Controller comparison
4. Performance tuning
5. Behavior cloning
6. State estimation
7. UI completion
8. Multi-vehicle
9. Sim-to-real

Algorithm expansion candidates

- EKF/UKF state estimation and covariance-aware failover.
- MPC or curvature-aware speed planning after baseline controllers.
- Graph-SLAM backend comparison and localization recovery.
- Cone classification/free-space fusion using RGB-D plus LiDAR.
- Nav2 waypoint comparison only where its assumptions fit the track.

Sequencing rule: Do not add algorithms merely to populate the menu. A profile enters the benchmark only after it has an executable launch, immutable manifest, readiness checks, safe stop and repeatable result.

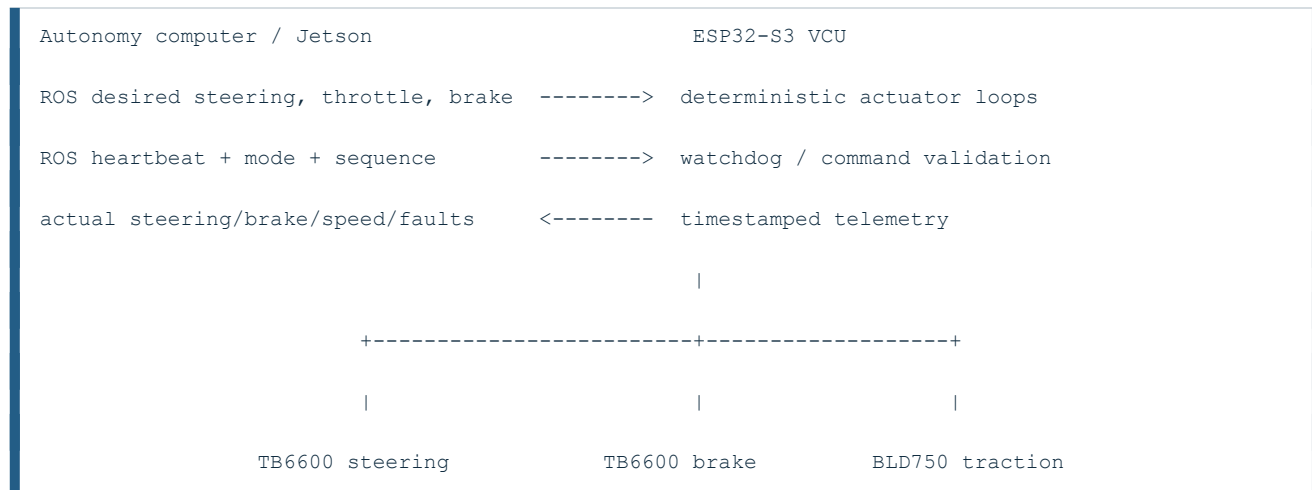
Simulation & Autonomy Engineering Report | Updated 28 September 2026

AUTOKART | SIMULATION TO REAL KART

Real-Kart Synchronization Plan

Target hardware context

ESP32-S3 vehicle controller; two NEMA 23/TB6600 axes for steering and brake; BLD750/BLDC traction through a 0-5 V PWM-to-voltage converter; steering/brake feedback and chassis sensing.



Required command architecture

- Simulation and real-kart adapters expose the same high-level command and feedback interfaces.
- The VCU, not ROS, owns pulse generation, limits, enable polarity, watchdog and emergency stop.
- Manual RC, autonomous ROS and simulator mirroring require explicit mode arbitration.
- Physical feedback drives the simulator only in a dedicated hardware-in-the-loop mode; simulated state must never overwrite safety-critical real feedback.
- Every command carries sequence, timestamp and timeout.

Validation stages

1. Software-in-the-loop without hardware motion.
2. VCU bench with disconnected actuators and synthetic feedback.
3. Hardware-in-the-loop with motors unloaded/lifted.
4. Manual RC mirrored into simulator.
5. Autonomy commands observed but not actuated.
6. Low-speed closed-course autonomous test with independent E-stop.

Safety boundary: Behavior cloning or simulator autonomy must never directly generate step pulses or analog motor voltage. It produces bounded vehicle-level targets; the VCU enforces hardware limits and can always reject or stop them.

Recommended Validation Sequence

Pre-motion gate

- 1. Validate JSON/YAML and exact generated manifest.
- 2. Start standalone FSDS and confirm RPC.
- 3. Start bridge and verify required topics/rates.
- 4. Validate TF tree and timestamps.
- 5. Open profile-specific RViz.
- 6. Confirm controller state is DISABLED.
- 7. Confirm exactly one control publisher.
- 8. Inspect path between boundaries.

Motion gate

Stage	Distance / objective	Pass condition
Launch	Leave start box.	Assist releases; state becomes TRACKING; no oscillation.
Straight	20-30 m.	Stable centerline and no cone approach.
First corner	One sector.	Path remains between boundaries; no stale brake.
Full lap	One complete loop.	Lap event, zero cones, result written.
Repeatability	10 reset trials.	At least 9 completions; median under target.

Failure capture

Record ROS bag, manifest, settings, controller status, path, command, speed, TF diagnostics, lap/cone telemetry and simulator log. Keep failed runs in the benchmark history.

Immediate next test: Run one fresh AMZ teach lap from the start pose, confirm loop acceptance and all map/centerline files, restart FSDS, launch replay disabled with RViz, then arm only after the loaded path overlays the track correctly.

Principal Source Files

Area	Principal source
Bridge	FSDS/ros2/src/fsds_ros2_bridge/src/*.cpp

Single Test	<code>single_test/src/config.cpp, prepare.cpp, session.cpp</code>
Teleop	<code>teleop/src/keyboard_teleop.cpp</code>
LiDAR autonomy	<code>autonomy/src/cone_detector.cpp, geometry.cpp, local_planner.cpp</code>
Persistent mapping	<code>autonomy/src/cone_map_builder.cpp; slam/src/route_recorder.cpp</code>
Controllers	<code>autonomy/src/pure_pursuit.cpp, route_planner.cpp</code>
Sensor odometry	<code>slam/src/sensor_odometry.cpp</code>
Behavior cloning	<code>demonstration_collector.cpp, inference_controller.cpp, training/train.py</code>
Benchmark UI	<code>FSDS/UE4Project/Plugins/AirSim/Source/SimHUD/</code>
Standalone build	<code>FSDS.sh; FSDS/scripts/fsds-package</code>

Key Git commits

Commit	Date	Change
3c850dcf	11 Aug	Working RGB-D D455 configuration.
16e85f31	11 Aug	VIO baseline.
db6c9ddf	12 Aug	VSLAM RTAB map patch.
8087e8c8	14 Aug	C++ LiDAR testing merge.
5346af0d	17 Aug	Single Test manifest.
92185f51	28 Aug	Behavior cloning pipeline.
2a2d32f8	29 Aug	Local autonomy.
1263ea26	15 Sep	Autonomy benchmark UI.
8813ff29	15 Sep	Working triangulation.
b3cfedcc	17 Sep	RViz custom profile in UI.
88e318a6	17 Sep	Standalone FSDS build.

Conclusion & Immediate Actions

The project has achieved a credible autonomy research platform: the simulator is repository-owned, ROS 2 sensor/TF integration works, several autonomy families share a common test contract, the benchmark UI runs in a standalone package, and historical mapping/autonomous results are preserved. The strongest engineering work is the infrastructure around reproducibility, safety-gated control and algorithm comparison.

The central gap is no longer “can FSDS connect to ROS?” It is “can one non-privileged autonomy profile repeatedly complete the track at useful speed?” The evidence currently says no. The route/SLAM path completed slowly in the past, local Delaunay did not complete, AMZ replay lacks active artifacts, and behavior cloning lacks a trainable multi-run dataset and model.

Next five actions

1. Fix the behavior dataset sample-count mismatch and improve validator failure reporting.
2. Generate one clean AMZ map/centerline package from a manual 44-50 s-quality lap.
3. Run conservative AMZ loaded-map replay with complete RViz evidence.
4. Establish three cone-free laps, then tune toward a median below 60 s.
5. Complete packaged UI click-through and add result-history visualization.

What has genuinely been achieved: A configurable FSDS autonomy benchmark foundation with working sensor integration, C++ autonomy components, teach/localize/replay infrastructure, behavior-cloning tooling, safety gates, manifests/results and a normal standalone simulator UI.

What must not yet be claimed: Production autonomy, repeatable sub-60-second laps, validated behavior-cloning control, complete AMZ replay, multi-vehicle racing, or safe real-kart autonomous actuation.

Report basis

Repository inspection on 28 September 2026; Git history through 88e318a6; generated result/map/dataset artifacts; package/runtime validation record; and the project development history. Static code presence is kept separate from simulator-observed behavior and repeatable performance evidence.